

Kaggriculture

A code guide to every Python file in the project

This document explains what each `.py` file in the repository does, how the agent decides what to do on every one of its 720 turns, and how the supporting tools were used to measure each change. It is written to be read on its own, without the code open beside it.

866	\$78,475	\$43,841	18×
LINES IN MAIN.PY	VS A PASSIVE OPPONENT	IN SELF-PLAY	VS PREVIOUS VERSION

1 The problem being solved

Before the code makes sense, the game does.

Kaggriculture is a two-player economic simulation. Each player runs a **10×10 farm** for a **30-day season** of **24 turns per day** — **720 decisions** in total. Both players sell into **one shared market**. The score is simply the money you hold at the end.

The three resources

RESOURCE	HOW IT WORKS	WHY IT MATTERS
Land	Start with one 25-tile quadrant; three more cost \$1,000 / \$2,000 / \$4,000	Tiles are where crops grow
Labour	Hire hands at a Fibonacci price that resets daily: 1, 1, 2, 3, 5, 8, 13...	Each unit takes <i>one</i> action per turn
Cash	Buys seeds, land, hands, animals	Scarce early, ample later

The market is the real opponent

Prices are not fixed. Every item has a price curve around a reference inventory of $I_0 = 10000$:

```
price(inventory) = base ± amplitude × f(|inventory - 10000|)
```

Sell into a market and its inventory rises, so **your own selling pushes the price down**. Meanwhile the town consumes goods continuously — shops unlock every 3 days (up to 8) and buy every 4 turns. Anything nobody sells drifts *below* I_0 and its price *rises* all season.

The shape function f decides whether a crop is a business or a one-shot prize:

CROP	CURVE	REVENUE @ 300 SOLD	VERDICT
WHEAT	log	\$6,313	Effectively uncapped — still \$20/unit
CARROT	sqrt	\$6,510	Mid
MELON	sq	\$26,627	Then \$1 forever — a one-shot prize
STRAWBERRY	linear	small	But 4 of 8 shops want it → scarcity premium reached \$333

The insight the whole agent is built on: because prices move with supply and the town drains inventory over time, the right question is never "what is this crop worth today?" but "**what will it be worth on the day it comes out of the ground?**"

Two hard limits that shape the code

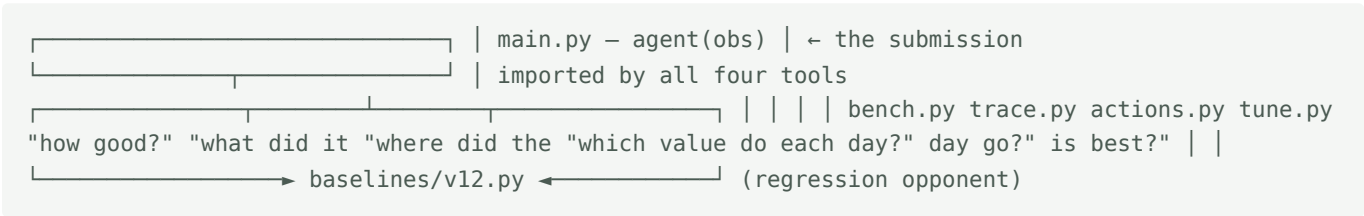
- The **shed holds 100 items**, and anything over that at end of day is **silently discarded**. Harvests sit in a worker's hands and only reach the shed at nightfall.
- The season stops **mid-day 29**, so day 29 never gets an end-of-day. Anything still being carried then is worth zero.

2 The files at a glance

Seven Python files. Exactly one of them is the competition entry.

FILE	LINES	ROLE	UPLOADED?
main.py	866	The agent. The entire submission.	Yes
bench.py	75	Parallel seeded benchmark — the scoreboard	No
tune.py	41	Sweeps one parameter across values	No
trace.py	61	Day-by-day narration of one game	No
actions.py	74	Profiles where the crew's day goes	No
baselines/v12.py	355	Frozen old agent, kept for regression checks	No
debug_wrapper.py	28	Early ad-hoc debug script, now redundant	No

How they relate



Only main.py is submitted. Kaggle loads it by executing the file and taking the *last callable defined in it* — `[v for v in env.values() if callable(v)][-1]`. That is `agent`, on line 860. This is why **nothing may be defined below agent**: a helper function added at the bottom would silently become the submitted agent.

Why one file with no dependencies

`main.py` imports only `math` and `os` from the standard library. Nothing else is available on Kaggle's runner, so the price curves, crop timings and town model are all **re-implemented from the engine source** rather than read from it. That duplication is deliberate: it is what lets the agent forecast prices instead of merely reacting to them.

3 main.py — the agent

One idea runs through the whole file: **price every possible action in dollars, then give each worker the best-paying job it can reach.**

That reframing is what makes the code tractable. There is no hand-written strategy, no "on day 5, plant melon". Everything — harvesting, watering, digging, walking to the shed, buying land, hiring — is converted into a dollar figure and thrown into one auction.

Layout of the file

LINES	PART	WHAT LIVES THERE
1–20	Docstring	The engine facts the design rests on
25–94	Game model	Crops, price curves, shops, land prices, animals
96–124	Tunables	~14 knobs, each overridable as an environment variable
127–275	Pure helpers	Price maths, crop plans, town demand, distances
278–857	<code>_decide(obs)</code>	The turn logic — everything happens here
860–866	<code>agent(obs)</code>	Thin <code>try/except</code> wrapper. Must be last.

3.1 The game model (lines 25–94)

Plain dictionaries transcribed from the engine. For example:

```
CROPS = {
    "WHEAT": {"seed": 10, "first_yield_day": 2, "max_yield_day": 4,
              "max_yield": 6, "ongoing": False},
    "MELON": {"seed": 80, "first_yield_day": 10, "max_yield_day": 12,
              "max_yield": 6, "ongoing": False},
    ...
}
```

`SHOPS` lists which shop buys what, and `EXPECTED_PULL` is computed once at import: since shops are drawn uniformly *with replacement*, this gives the expected consumption of each item per shop per tick. Single-product shops pull double.

3.2 The tunables (lines 96–124)

```
def _tune(name, default):
    raw = os.environ.get("KAG_" + name)
    return type(default)(raw) if raw is not None else default

MAX_UNITS = _tune("MAX_UNITS", 12)
MOVE_COST = _tune("MOVE_COST", 7.0)
PORTER_FILL = _tune("PORTER_FILL", 0.6)
```

Small, but it is what made the project measurable. Every constant can be overridden from the shell, so `tune.py` can sweep any of them without editing a line of code. **Every value in the table below was found by measurement, not judgement.**

PARAMETER	VALUE	WHAT THE SWEEP SHOWED
MAX_UNITS	12	8→\$66k, 10→\$69k, 12→\$69k , 14→\$63k
TILES_PER_UNIT	6.0	5→\$48k (over-hires), 6→\$71k, 8→\$66k
MOVE_COST	7.0	Flat from 3–7; 10→\$48k, 15→\$46k
SEED_RATIO	6	4→\$62k, 6→\$77k , 10→\$74k, 20→\$68k
LAND_BUFFER	800	200→\$50k (starves seed buying); ≥400 all equal
PORTER_FILL	0.6	+\$2,202 versus disabled
MAX_GEESE	0	4→-\$10k, 8→-\$17k, 16→-\$25k

3.3 The pure helpers (lines 127–275)

Six functions do the economic reasoning. They are pure — same input, same output — which is why they can be reasoned about and tested in isolation.

FUNCTION	WHAT IT ANSWERS
<code>market_price(item, inv)</code>	An exact port of the engine's price curve. The foundation for everything else.
<code>_build_crop_plans()</code>	For each crop, the best age to harvest at and the yield reachable by then. Runs once at import.
<code>daily_town_demand(shops, day, horizon)</code>	How much the town will drain per day — <i>including shops that have not unlocked yet</i> .
<code>batch_revenue(item, inv, n)</code>	Revenue from selling <i>n</i> units, correctly accounting for the price falling as you go.
<code>sellable_count(item, inv, have, reserve)</code>	How many to sell before the marginal price drops below the reserve.
<code>step_toward(x, y, tx, ty)</code>	One step. No pathfinding needed — nothing blocks movement.

`_build_crop_plans()` is a good example of derivation replacing guesswork. Watering only adds yield inside a window `[[max_yield_day/2], max_yield_day]`, so the function computes the earliest age at which a plant already carries its full yield:

```
window_start = (cd["max_yield_day"] + 1) // 2
units = min(cd["max_yield"], 1 + (cd["max_yield_day"] - window_start + 1))
```

The results — WHEAT harvest at age 4 for 4 units, MELON at 10 for 6, STRAWBERRY at 16 for 4 — were then verified against real games.

4 `_decide()` — one turn, step by step

The 580-line core. It runs 720 times per game and takes about 4.6 ms, against a 1,000 ms limit.

1 Scan the farm (line 305)

Walk all 100 tiles once, sorting them into empties, weeds, and plants. At the same time total the **pipeline**: everything of each type we own but have not yet sold — growing, in the shed, and in workers' hands. This number is what stops the agent flooding its own market later.

2 Value the crops (line 369)

The most important idea in the file. A planting is priced against the inventory it will *face at harvest*, not today's:

```
drawdown = demand_over(cycle)[crop] * cycle
start    = today_inventory - drawdown + our_pipeline
profit   = batch_revenue(crop, start, plan["units"]) - seed_cost
```

The `drawdown` term is what discovers scarcity premiums; the `pipeline` term is what prevents self-flooding.

3 Rank the crops (line 405)

Profit alone is the wrong ranking, because *which resource is scarce changes during the game*. On day 0 land is free and cash is tight, so a \$100 strawberry seed is a trap. By day 15 cash is ample and every tile-day counts — the opposite answer. Charging seed cost against cash-per-free-tile interpolates between the two automatically:

```
cash_per_tile = (money - CASH_FLOOR) / len(empties)
score = profit / (cycle * (1 + seed_cost / cash_per_tile))
```

This single change was worth +\$3,700 per game.

4 Build the job list (line 462)

Every possible action becomes a priced entry. A harvest is worth its produce plus half the value of the tile it frees:

```
add(yield_units * price_of(crop) + tile_unlock_value * 0.5, x, y, ["HARVEST"])
```

Watering is priced twice over — once for the yield it adds inside the window, once for the death it prevents outside it. Digging a weed is priced at the value of the tile it returns. Each job also carries a `need` (an item the worker must already hold), an optional `unit` pin, and a `key` naming what two jobs may not share this turn.

5 Shed runs (line 598)

SHIPPED THIS SESSION Goods only reach the shed at nightfall, and overflow past 100 is destroyed. So once the day's haul is outgrowing the shed, workers carry produce there mid-day where the sell orders can drain it. Detailed in §5.

6 Assign workers to jobs (line 648)

Rank *every* (worker, job) pair and take them greedily:

```
pairs.append((value / (dist + 1.0), unit_idx, job_idx))
pairs.sort(key=lambda p: -p[0])
```

Dividing by distance means a worker prefers a decent job at its feet over a great one across the farm. Jobs whose value does not survive the walk ($\text{value} - \text{dist} \times \text{MOVE_COST} \leq 0$) are never offered at all. Once a job is taken its `key` is claimed so nobody duplicates it.

7 Turn assignments into actions (line 709)

A worker not yet standing on its target emits a movement step; one that has arrived emits the operation.

`PLANT` is special — the crop is chosen at *execution* time from a decrementing seed counter, because the engine **discards every planting of a crop** if you queue more than you hold seeds for.

8 Market orders (line 733)

Orders resolve in list order, so selling comes first and funds everything after it: `SELL` → `BUY_LAND` → `HIRE` → `BUY_SEED`. The sell logic holds stock back while the price is below $\text{RESERVE_FRAC} \times \text{base}$, and relaxes that reserve as the season closes (0.15 with four days left, 0 on the last).

Two invariants that must not be broken

- **Never queue more `PLANT` ops for a crop than seeds held** — the engine drops *all* of that crop's plantings for the turn.
- **Never plant beyond $\text{crew} \times \text{PLANTS_PER_UNIT}$** — an unwatered plant is not merely a wasted seed, it becomes a weed that then costs a DIG to remove.

The wrapper (line 860)

```
def agent(obs):
    try:
        return _decide(obs)
    except Exception:
        if DEBUG:
            raise
        return {"farmer": ["PASS"], "hands": [], "market": []}
```

On Kaggle a raised exception forfeits the game, so the wrapper degrades to a safe no-op instead. During development `KAG_DEBUG=1` disables the catch — without it, a crashing agent is indistinguishable from a merely bad one.

5 The two experiments this session

One shipped, one was measured and switched off. Both are results.

Mid-day porter runs SHIPPED · +\$2,202

The problem. Harvested goods sit in a worker's hands and only reach the shed at end of day. The shed caps at 100 items and everything past that is **silently discarded**. On strong days the farm was out-producing its own storage and quietly throwing away the surplus.

The fix. Once the shed is projected past 60% full, workers make delivery runs *during* the day, so the sell orders can drain it before nightfall:

```
crowded = shed_fill + incoming > SHED_CAP * PORTER_FILL
if last_day or (crowded and shed_fill < SHED_CAP):
    worth = sum(market_price(i, minv[i]) * q for i, q in inv.items() ...)
    add(worth + 1e6 if last_day else worth * 0.6, tx, ty, ["DROP", idx],
        unit=idx, key=(tx, ty, "DROP"))
```

The job is *pinned* to the worker holding the goods (`unit=idx`) — nobody else can carry someone's hands to the shed. On the last day the value is inflated by `1e6` to force the run, since unsold carried goods score zero.

48 PAIRED SEEDS VS PASS	MEAN	MEDIAN	MINIMUM
Porter runs on	\$78,475	\$79,689	\$48,171
Porter runs off	\$76,273	\$76,503	\$43,547
Difference	+\$2,202	+\$3,186	+\$4,624

It lifts the *floor* harder than the mean, which is exactly the expected signature: it pays in precisely those games where production overran the shed.

A lesson about sample size. Measured on 16 seeds this change looked *negative*: −\$889 on one seed window, +\$2,568 on another. Only at $n=48$ did the real effect emerge. Game-to-game variation is roughly \$11,000, so small samples can and do point the wrong way.

The goose / egg engine BUILT, MEASURED, DISABLED

The previous handoff called livestock "the largest single remaining lever". It was implemented in full — coops built, birds bought, fetched from the shed, placed, fed, cared for, harvested, fertilizer collected. Then it was measured:

MAX_GEESE	0	4	8	16
Mean score	\$79,472	\$69,309	\$62,858	\$54,027

Why the original estimate was wrong. The case for geese was "−\$80 per tile-day versus wheat's ~\$22" — arithmetic per *tile*. But tiles are not the binding constraint; **worker-actions are**. A goose grosses about \$150/day, but collecting it takes FEED + CARE + COLLECT + part of a HARVEST, each on the coop tile, with a walk between every one. At roughly seven actions per bird per day, sixteen birds consume more than half the crew's entire day — and traces showed it still only managing to feed 7–11 of them.

It ships behind `MAX_GEESE = 0`, complete and ready, should the walking problem ever be solved. The alternative explanation — that the `G00SE_UPKEEP` term was double-charging crew capacity — was tested directly and ruled out (a −112 difference, i.e. noise).

Why this counts as a good outcome. The feature works correctly; it simply does not pay. Knowing that with measured numbers is worth more than shipping it on the strength of a plausible-sounding estimate — and the diagnosis (*actions, not tiles, are scarce*) is now the single most useful fact guiding what to do next.

6 The four measurement tools

None of these are submitted. They exist so that no change was ever shipped on opinion.

`bench.py` **THE SCOREBOARD** — 75 lines

Plays many games in parallel on **fixed seeds** and reports the distribution.

```
uv run bench.py --opp pass -n 48 --workers 16          # ~29 seconds
uv run bench.py --opp baselines/v12.py -n 10 --swap    # regression, both seats
```

Three design choices matter:

- **Fixed seeds.** Seeds run from `--seed0` upward, so two versions face identical games. This is *paired* comparison, and it removes most of the noise.
- **--swap.** Replays every seed with the seats exchanged, cancelling any first-player advantage.
- **A `ProcessPoolExecutor`.** One game costs ~4.6 s of CPU; 16 workers give ~1.7 games/second.

It prints mean, median, min, max, standard deviation and winrate. The **minimum** matters as much as the mean — the difference between a strategy that is good on average and one that is reliable.

Always benchmark against `pass`, never `random`. The built-in random agent calls `random.Random()` with no seed, so it behaves differently every run — about 3.5% of noise, enough to drown a real \$2,000 improvement.

`tune.py` **THE PARAMETER SWEEP** — 41 lines

```
uv run tune.py MAX_UNITS 10 12 14

baseline $ 69,000          ← output format; the
MAX_UNITS=10 $ 69,000      -0    measured result of this
MAX_UNITS=12 $ 69,000      +0    sweep was 10→$69k, 12→$69k,
MAX_UNITS=14 $ 63,000      -6,000 14→$63k
```

It scores a baseline, then re-runs `bench.py` as a subprocess once per value with `KAG_<NAME>` set in the environment, always on the same seeds. Every number in the tunables table of §3.2 came out of this. It is short only because `_tune()` in `main.py` did the hard part.

`trace.py` **THE NARRATOR** — 61 lines

Wraps `agent`, and at one chosen hour each day prints a line of state — the shape of the output is:

```
D<day> $<money> hands=<n> land=<quadrants> free=<empty tiles> weed=<n>
birds=<placed>/<coops> plants={crop: count} seeds={crop: count}
shed={item: qty} px={item: current market price}
```

This is the tool for *why*. A benchmark says a change lost \$5,000; a trace shows the money pinned at \$185 for six days because the agent bought birds before it bought land.

`actions.py` **THE PROFILER** — 74 lines

Counts every action the crew issues, grouped by operation. Written specifically to answer why the geese lost money — and it produced the session's most important number:

OPERATION	SHARE OF ALL WORKER-ACTIONS
MOVE (walking)	51.7%
PASS (idle)	11.5%
FEED	2.0%
CARE	1.6%

More than half of everything the crew does is *walking*. That single figure explains the goose result, and it makes routing the biggest remaining opportunity in the project.

Reach for `actions.py` whenever a change looks economically sound but does not pay. A job that prices well per action can still lose money once the walk to it is counted.

`baselines/v12.py` — 355 lines

The previous agent, frozen. It is never edited — it exists so that `--opp baselines/v12.py --swap` can confirm no refactor has quietly regressed. Current agent: **20 wins from 20** across both seats.

`debug_wrapper.py` — 28 lines

The earliest debugging script, from before `trace.py` existed. Fully superseded and safe to delete; kept only for history.

7 Results, and what remains

Where the agent stands

OPPONENT	GAMES	MEAN	MEDIAN	MINIMUM	WINRATE
random	16	\$81,166	\$80,938	\$72,201	16/16
pass (deterministic)	48	\$78,475	\$79,689	\$48,171	48/48
v12, both seats	20	\$74,635	\$73,885	\$55,793	20/20
Self-play , both seats	16	\$43,841	\$44,334	\$23,881	mirror

The previous version scored **\$4,464**. The current agent is about **18×** that, and uses 4.6 ms of its 1,000 ms per-turn budget — so there is a great deal of compute headroom still unspent.

The self-play row is the one to quote — \$78,475 is a benchmark, not a forecast. The first three opponents barely sell anything, so our agent alone drains the town's demand and collects every scarcity premium. Against an opponent that genuinely competes for the shared market the score falls **44%**, to about \$44k.

Its winrate column is blank because the figure would mislead: the agent is deterministic, so against a copy of itself it plays a *mirror match* and finishes on identical dollars. About two-thirds of seeds tie to the dollar, and `bench.py` counts a tie as a non-win.

The changes that produced the gain

CHANGE	WORTH
Rewriting valuation to forecast harvest-day prices	bulk of the gain
Ranking crops against whichever resource is scarce	+\$3,700
Modelling the town's drawdown (scarcity premiums)	+\$2,000
Mid-day porter runs	+\$2,202
Skipping redundant waterings outside the yield window	large, frees crew time

What is still open

- Price the opponent's supply.** Self-play costs 44% of the score, and `crop_profit` is the prime suspect: it starts from `today - town_drawdown + our_pipeline` and has **no term for what the opponent sells into the same inventory**. Every crop is overvalued in a contested game, and the bias is worst on exactly the crops the agent favours most — a scarcity premium is large precisely because *nobody* is selling, which is the assumption a real rival breaks. Worth roughly \$35,000 a game, and the only item on this list measured against a real opponent.
- Cut the walking.** 51.7% of actions are moves and 11.5% are idle. Assignment is greedy per-turn with no notion of a route, so workers criss-cross the farm. The biggest remaining lever on the crop engine *and* the precondition for livestock ever paying.
- Land timing.** Quadrants 3 and 4 still arrive around day 11 because cash is tied up in melon seeds until the day-10 harvest. Reserving toward the next land price is likely worth thousands.
- The panic-dump rule is mispriced** (*a known defect*). When the shed is nearly full the reserve drops to 0.05 and the agent sells almost anything at almost any price — traces showed melon dumped at **\$4** against a \$250 base. Overflow really is destroyed, so the rule is right in principle, but it should dump the *cheapest* items rather than everything.

Submitting

Kaggle → Submit to Competition → File Upload → Browse Files → `main.py`

A single file, not an archive. The zip option exists for agents split across several modules, which would then need `main.py` at the archive root. Verified through Kaggle's exact loading path: loaded as a file rather than an imported module it scores \$77,114 over 8 games, matching the module numbers.

Running `python main.py` prints nothing, and that is correct. It is the agent *module*, not a program — it defines constants and functions and exits. Silence with exit code 0 means the file is healthy. To watch it actually play, use `trace.py`; to score it, use `bench.py`.

Kaggriculture · branch `KA-agent` · every figure in this document was measured with `bench.py` over fixed seeds, not estimated. Companion documents: `PROJECT_STATUS.md` (status and runbook) · `kaggriculture_handoff.md` (engine internals and design rationale).